



UNIVERSITÉ DE HASSIBA BENBOUALI CHLEF
FACULTÉ DES SCIENCES EXACTES ET D'INFORMATIQUE
DÉPARTEMENT D'INFORMATIQUE
LICENCE 3



DONNÉES SEMI-STRUCTURÉES

CHAPITRE 04

S. Medjroud

samedjrd@gmail.com

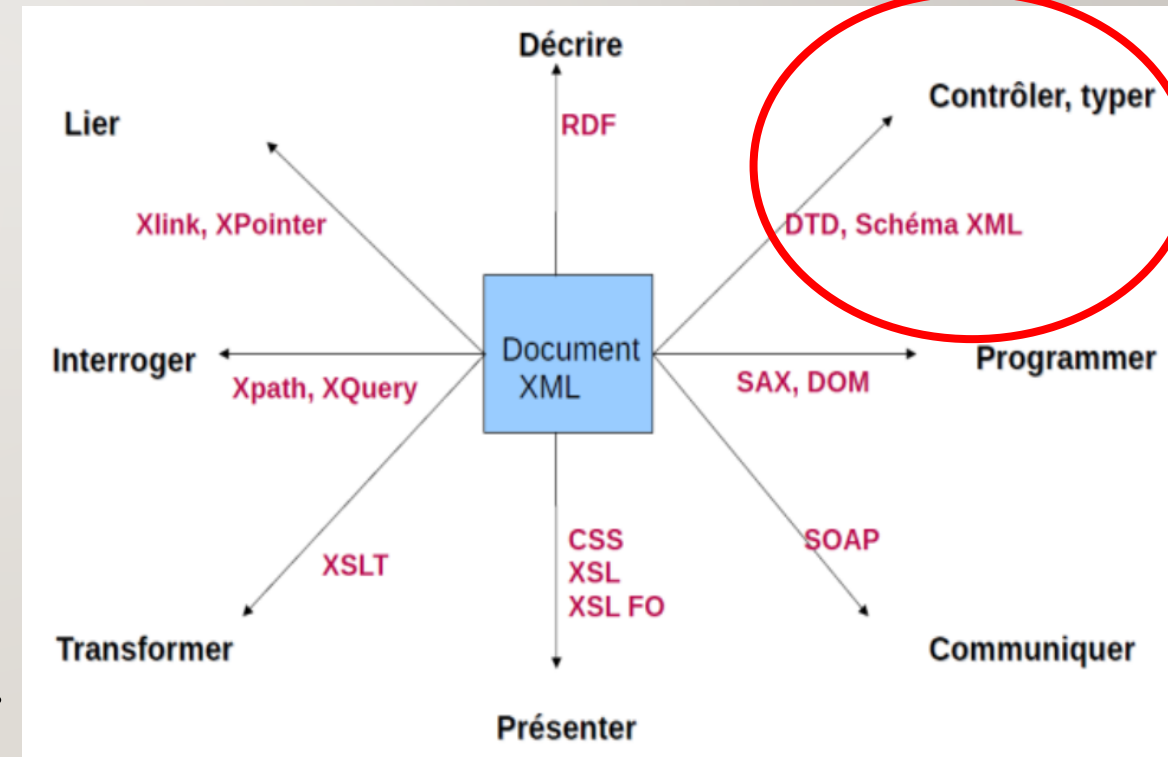
2025-2026

CHAPITRE 04: GALAXIE XML

“Galaxie XML” means: all technologies built **around XML** to:

- navigate
- transform
- process
- link data

➤ Think of XML as the **data**,
and the rest as **tools to use that data**.



CHAPITRE 04: GALAXIE XML

4.1. Les chemins: XPATH

4.1.1. Principes

4.1.2. Les axes

4.1.3. Les filtres

4.1.4 Les prédicats

4.1. LES CHEMINS : XPATH

- **XML Path** ou **XPath** est un langage qui permet de sélectionner un ou plusieurs **éléments** dans un document **XML**.
- Un document **XML** peut être représenté sous la forme d'un **arbre** composé de **nœuds**.
- **XPath**, grâce à une notation particulière, permet de **localiser** précisément un **composant de l'arbre**.

DEFINITION

- XPath is a language used to **navigate inside an XML document** and select elements.
- It works like **file system paths** (folders).

XML EXAMPLE / TREE STRUCTURE

</> XML

```
<directory>
  <person>
    <name>Ali</name>
    <firstname>Ahmed</firstname>
    <phone>123456</phone>
  </person>
</directory>
```



```
directory
├── person
│   ├── name
│   ├── firstname
│   └── phone
```

XPATH VOCABULARY (1/5)

- **I. Parent**

The **parent** is the node directly above another node.

Example:

- ❑ person is the parent of name, firstname, and phone
- ❑ directory is the parent of person

```
</> XML
```

```
<directory>  
  <person>  
    <name>Ali</name>  
    <firstname>Ahmed</firstname>  
    <phone>123456</phone>  
  </person>  
</directory>
```

XPATH VOCABULARY (2/5)

- **2. Child**

A **child** is a node directly below another node.

Example:

name, firstname, and phone

are children of person

`</> XML`

```
<directory>
  <person>
    <name>Ali</name>
    <firstname>Ahmed</firstname>
    <phone>123456</phone>
  </person>
</directory>
```


XPATH VOCABULARY (3/5)

- **3. Ancestor**

An **ancestor** is any node above (parent, grandparent, etc.)

Example (for phone):

parent → person

grandparent → directory

So ancestors are: person, directory

</> XML

```
<directory>
  <person>
    <name>Ali</name>
    <firstname>Ahmed</firstname>
    <phone>123456</phone>
  </person>
</directory>
```

XPATH VOCABULARY (4/5)

- **4. Descendant**

A **descendant** is any node below (children, grandchildren...)

Example (for directory):

person, name, firstname, phone

```
</> XML
```

```
<directory>  
  <person>  
    <name>Ali</name>  
    <firstname>Ahmed</firstname>  
    <phone>123456</phone>  
  </person>  
</directory>
```

XPATH VOCABULARY (5/5)

- **5. Sibling**

Siblings are nodes with the same parent.

Example:

name, firstname, and phone are siblings
(because they share the same parent: person)

</> XML

```
<directory>
  <person>
    <name>Ali</name>
    <firstname>Ahmed</firstname>
    <phone>123456</phone>
  </person>
</directory>
```

EXAMPLE

- Si je veux récupérer par exemple

le numéro de fixe :

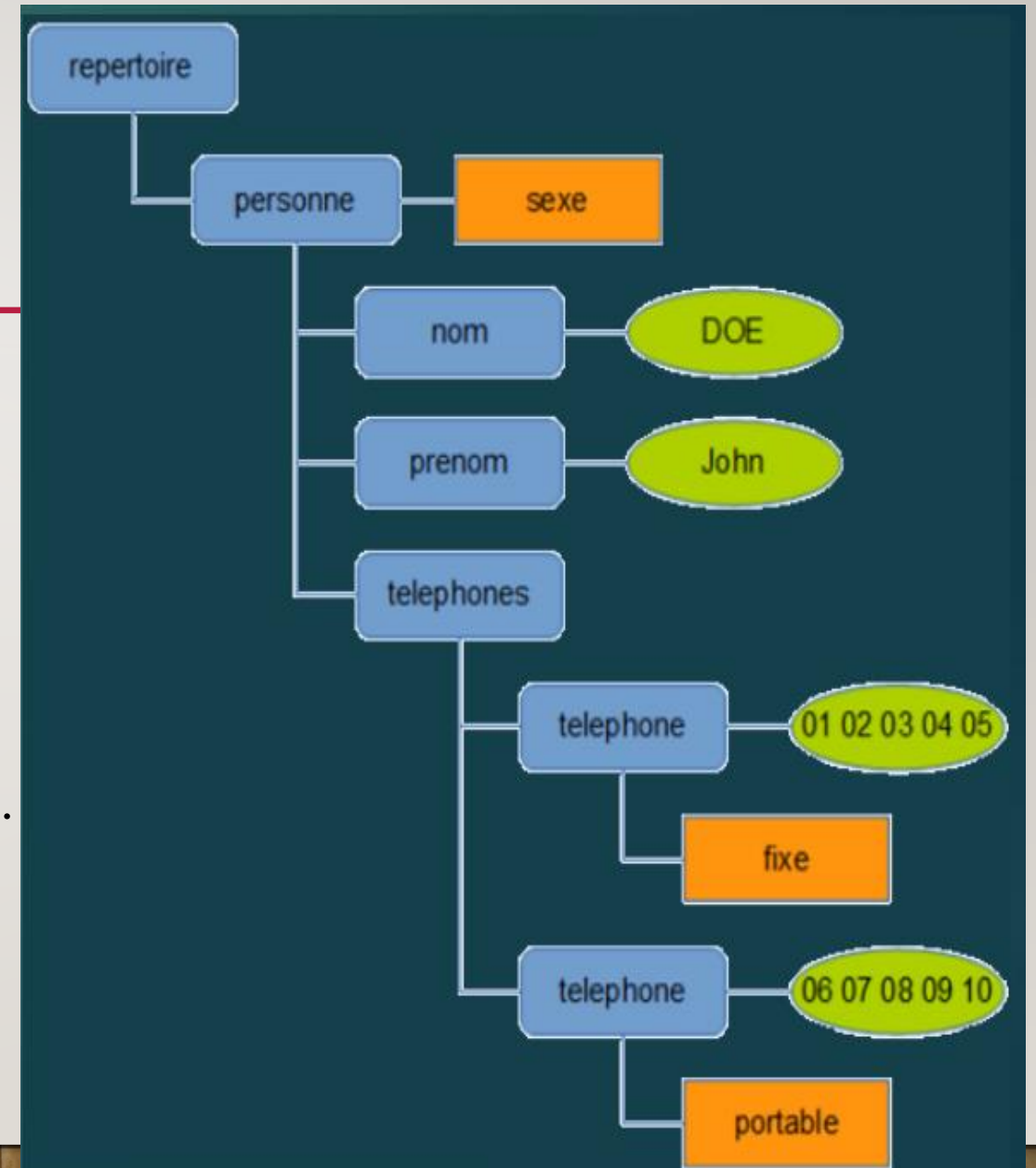
Etape 1 : nœud "repertoire". (racine de l'arbre)

Etape 2 : descendre au nœud enfant "personne".

Etape 3 : descendre au nœud enfant "telephones".

Etape 4 : descendre au nœud enfant "telephone"
dont l'attribut est "fixe".

/étape1/étape2/étape3/étape4



LOCALISER LES DONNÉES

Axe I :: filtre [prédicat]

1. Un axe: définir le sens de la recherche, se diriger vers un nœud enfant ou remonter vers un nœud parent.
2. Un filtre : nom du nœud ou un type de nœud.
3. Un ou plusieurs prédicats (facultatif): agissent comme un filtre.

AXIS (LES AXES) : DEFINITION

➤ Axis = **direction in the XML tree**

➤ It tells XPath **where to go**

- Examples:

❑ child::book → go to children

❑ parent::bookstore → go up

❑ descendant::title → go down (all levels)

AXIS (LES AXES) : EXAMPLE

- /bookstore/child::book
- ❑ Start at bookstore
- ❑ Move to its children (book)

Result: all <book>

➤ Axis = **navigation only**, no condition

```
<bookstore>
  <book id="1" category="web">
    <title>XML Basics</title>
    <author>Ali</author>
    <price>50</price>
  </book>

  <book id="2" category="AI">
    <title>Machine Learning</title>
    <author>Sara</author>
    <price>100</price>
  </book>
</bookstore>
```

AXIS (LES AXES)

Nom de l'axe	Description
ancestor	oriente la recherche vers les ancêtres du nœud courant
ancestor-or-self	oriente la recherche vers le nœud courant et ses ancêtres
attribute	oriente la recherche vers les attributs du nœud courant
child	oriente la recherche vers les enfants du nœud courant
descendant	oriente la recherche vers les descendants du nœud courant
descendant-or-self	oriente la recherche vers le nœud courant et ses descendants
following	oriente la recherche vers les nœuds suivant le nœud courant
following-sibling	oriente la recherche vers les frères suivants du nœud courant
parent	oriente la recherche vers le père du nœud courant
preceding	oriente la recherche vers les nœuds précédant le nœud courant
preceding-sibling	oriente la recherche vers les frères précédents du nœud courant
self	oriente la recherche vers le nœud courant

Requête :

Récupérer le **pays de domiciliation** de John DOE.

Expression xpath:

- **Etape 1** : descendre au nœud "**repertoire**".
- **Etape 2** : descendre au nœud "**personne**".
- **Etape 3** : descendre au nœud "**adresse**".
- **Etape 4** : descendre au nœud "**pays**".

Chemin absolu:

/étape1/étape2/étape3/étape4



/child::repertoire/child:personne/child:adresse/child:pays

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
```

```
<repertoire>
```

```
  <personne sexe="masculin">
```

```
    <nom>DOE</nom>
```

```
    <prenom>John</prenom>
```

```
    <adresse>
```

```
      <numero>7</numero>
```

```
      <voie type="impasse">impasse du chemin</voie>
```

```
      <codePostal>75015</codePostal>
```

```
      <ville>PARIS</ville>
```

```
      <pays>FRANCE</pays>
```

```
    </adresse>
```

```
    <telephones>
```

```
      <telephone type="fixe">01 02 03 04 05</telephone>
```

```
      <telephone type="portable">06 07 08 09 10</telephone>
```

```
    </telephones>
```

```
    <emails>
```

```
      <email type="personnel">john.doe@wanadoo.fr</email>
```

```
      <email type="professionnel">john.doe@societe.com</email>
```

```
    </emails>
```

```
  </personne>
```

```
</repertoire>
```


Requête :

Récupérer le **pays de domiciliation** de John DOE. Mais notre point de départ sera le nœud "**telephones**".

Expression xpath:

- **Etape 1** : remonter au nœud frère "**adresse**"
- **Etape 2** : descendre au nœud "**pays**".

Chemin relatif:

/étape1/étape2



preceding-sibling::adresse/pays

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<repertoire>
  <personne sexe="masculin">
    <nom>DOE</nom>
    <prenom>John</prenom>
    <adresse>
      <numero>7</numero>
      <voie type="impasse">impasse du chemin</voie>
      <codePostal>75015</codePostal>
      <ville>PARIS</ville>
      <pays>FRANCE</pays>
    </adresse>
    <telephones>
      <telephone type="fixe">01 02 03 04 05</telephone>
      <telephone type="portable">06 07 08 09 10</telephone>
    </telephones>
    <emails>
      <email type="personnel">john.doe@wanadoo.fr</email>
      <email type="professionnel">john.doe@societe.com</email>
    </emails>
  </personne>
</repertoire>
```

- ❑ L'axe child est l'axe par défaut donc possible de ne pas l'écrire.

`/child::repertoire/child:personne/child::adresse/child:pays`



`/repertoire/personne/adresse/pays`

- ❑ remplacer l'expression " `/descendant-or-self::node()/pays`" ➡ `//pays`.

- ❑ remplacer `/repertoire/personne/self::node()` ➡ `/repertoire/personne/`.

- ❑ remplacer " `parent::node()` " ➡ `..`

FILTER (LES FILTRES): DEFINITION

□ “A filter is a way to keep only the elements we want.”

□ Example :

➤ “We have many books, but we don’t want all of them.”

➤ “We apply a filter to select only some books.”

FILTER (LES FILTRES): EXAMPLE

❑ `//book[price > 50]`

❑ “Here, we filter books to keep only those with price greater than 50.”

FILTER (LES FILTRES)

Nom	Description
nom du nœud	oriente la recherche vers le nœud dont le nom a explicitement été spécifié
*	oriente la recherche vers tous les nœuds
node()	oriente la recherche vers tous les types de nœuds (éléments, commentaires, attributs, etc.)
text()	oriente la recherche vers les nœuds de type texte
comment()	oriente la recherche vers les nœuds de type commentaire

PREDICATE (LES PRÉDICATS): **DEFINITION**

- ❑ “In XPath, this filter has a name: it is called a *predicate*.”
- ❑ Predicate = **condition inside []**
- ❑ It tells XPath **which nodes to keep**

PREDICATE (LES PRÉDICATS): **EXAMPLES**

- ❑ `/bookstore/book[1]` → first book
- ❑ `/bookstore/book[price > 50]` → expensive books
- ❑ `//book[@category="AI"]` → AI books

LES PRÉDICATS

Nom du prédicat	Description
attribute	permet d'affiner la recherche en fonction d'un attribut
count()	permet de compter le nombre de nœuds
last()	permet de sélectionner le dernier nœud d'une liste
position()	permet d'affiner la recherche en fonction de la position d'un nœud

Requête 1:

récupérer le nœud contenant **le numéro de téléphone fixe** de John DOE.

Deux Expressions xpath possible:

Axe1 :: filtre [prédicat]

1. Nous allons sélectionner tous les descendants du nœud racine et filtrer sur la valeur de l'attribut **type**.

/descendant :: * [attribute::type="fixe"]

2. Préciser le chemin complet

/repertoire/personne/telephones/telephone[attribute::type="fixe"]

Requête 2:

sélectionnant les numéros de téléphones qui ne sont pas des numéros de téléphones fixes.

Expression xpath :

/repertoire/personne/telephones/telephone[attribute::type != "fixe"]

Ou:

/repertoire/personne/telephones/telephone[not(attribute::type = " fixe ")]

Requête 3:

sélectionner le **premier numéro de téléphone** de John DOE.

Expressions xpath :

/repertoire / personne / telephones / telephone [position()=1]

Requête 4:

sélectionner le dernier nœud "téléphone" de la liste de John:

Expressions xpath :

/repertoire / personne / telephones / telephone [last()=1]

Abréviation

/repertoire/personne/telephones/telephone [attribute::type = "fixe "]

Est équivalente à:

/repertoire/personne/telephones/telephone[@type="fixe"]

EXEMPLE AVEC EDITIX

The screenshot displays the Editix XML editor interface with three main panels:

- Left Panel (XPath):** Shows the XPath expression `/repertoire/personne/emails/email[attribute::type="professionnel"]` and its results. The results table is as follows:

Node	Value
email	john.doe@societe.com
email	contact@poppins.fr

- Middle Panel (Tree View):** Displays the XML document structure. The selected node is `email (1) professionnel` under `personne (5) feminin`.
- Right Panel (XML Document):** Shows the XML content with line numbers. The document structure is as follows:

```
41 <email type="professionnel">john.doe@societe.com</email>
42
43 </emails>
44
45 </personne>
46
47
48 <!-- Marie POPPINS -->
49
50 <personne sexe="feminin">
51
52   <nom>POPPINS</nom>
53
54   <prenom>Marie</prenom>
55
56   <adresse>
57
58     <numero>28</numero>
59
60     <voie type="avenue">avenue de la république</voie>
61
62     <codePostal>13005</codePostal>
63
64     <ville>MARSEILLE</ville>
65
66     <pays>FRANCE</pays>
67
68   </adresse>
69
70   <telephones>
71
72     <telephone type="professionnel">04 05 06 07 08</telephone>
73
74   </telephones>
75
```


BASIC SYNTAX

Expression	Meaning
/	from root
//	anywhere
.	current node
..	parent
@	attribute

EXAMPLES : / (ROOT SELECTION)

- Starts from the root of the document

➤ /bookstore

Selects the root element <bookstore>

➤ /bookstore/book

Selects all <book> elements directly under bookstore



```
<bookstore>
  <book id="1" category="web">
    <title>XML Basics</title>
    <author>Ali</author>
    <price>50</price>
  </book>
```



```
<book id="2" category="AI">
  <title>Machine Learning</title>
  <author>Sara</author>
  <price>100</price>
</book>
</bookstore>
```

EXAMPLES : // (ANYWHERE)

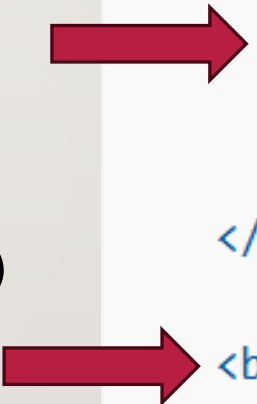
- Select nodes **anywhere in the document**

➤ //title

Selects ALL <title> elements (no matter where)

➤ //book

Selects all <book> elements



```
<bookstore>
  <book id="1" category="web">
    <title>XML Basics</title>
    <author>Ali</author>
    <price>50</price>
  </book>
  <book id="2" category="AI">
    <title>Machine Learning</title>
    <author>Sara</author>
    <price>100</price>
  </book>
</bookstore>
```

The diagram illustrates the use of XPath expressions. A red arrow points from the text '//title' to the first <title> element in the XML document. Another red arrow points from the text '//book' to the second <book> element in the XML document.

EXAMPLES : . (CURRENT NODE)

Refers to the **current node**

Example:

➤ .

Selects the current node

➤ ./title

Select title inside current node

EXAMPLES : .. (PARENT)

- Move to parent node

➤ //title/..

From <title> go back to <book>



```
<bookstore>
  <book id="1" category="web">
    <title>XML Basics</title>
    <author>Ali</author>
    <price>50</price>
  </book>

  <book id="2" category="AI">
    <title>Machine Learning</title>
    <author>Sara</author>
    <price>100</price>
  </book>
</bookstore>
```

EXAMPLES : @ (ATTRIBUTES)

- Select attributes

➤ //book/@id

Select all id attributes

➤ //book[@category="AI"]

Select book where category = AI

```
<bookstore>
```



```
<book id="1" category="web">  
  <title>XML Basics</title>  
  <author>Ali</author>  
  <price>50</price>  
</book>
```



```
<book id="2" category="AI">  
  <title>Machine Learning</title>  
  <author>Sara</author>  
  <price>100</price>  
</book>  
</bookstore>
```


EXAMPLES : * (WILDCARD)

Select ANY element

➤ /bookstore/*

Select all children of bookstore

➤ /**

Select ALL elements in XML

```
<bookstore>
  <book id="1" category="web">
    <title>XML Basics</title>
    <author>Ali</author>
    <price>50</price>
  </book>
  <book id="2" category="AI">
    <title>Machine Learning</title>
    <author>Sara</author>
    <price>100</price>
  </book>
</bookstore>
```

EXAMPLES : | (OR UNION)

Combine multiple paths

➤ //title | //author

Select all titles AND authors

```
<bookstore>
  <book id="1" category="web">
    <title>XML Basics</title>
    <author>Ali</author>
    <price>50</price>
  </book>

  <book id="2" category="AI">
    <title>Machine Learning</title>
    <author>Sara</author>
    <price>100</price>
  </book>
</bookstore>
```

SUMMARY

Concept	Role
Axis	Direction (how you move in the tree)
Predicate	Condition (which nodes you keep)
Filter	General idea of restricting results (often = predicate)

CHAPITRE 04: GALAXIE XML

4.2. Les feuilles de style et traitements XSL

XSL (eXtensible Stylesheet Language)

QU'EST-CE QUE XSL EXACTEMENT ?

- **XSL (eXtensible Stylesheet Language)** est un langage qui permet de :
 - Transformer un document XML
 - Contrôler son affichage
 - Générer d'autres formats (HTML, PDF, TXT...)
- Contrairement à CSS :
 - ❑ **CSS** → change seulement le style
 - ❑ **XSL** → peut **modifier la structure + le contenu**

LES 3 COMPOSANTS DE XSL

1. XSLT (XSL Transformations) — le cœur
2. XPath — le moteur de sélection
3. XSL-FO (Formatting Objects)

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

- C'est la partie **la plus importante**.
- XSLT permet de :
 - ❖ Transformer XML → HTML
 - ❖ Transformer XML → XML
 - ❖ Transformer XML → texte

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

- Principe clé : Transformation basée sur des règles
- ❑ XSLT fonctionne avec des **templates (modèles)** :
- ❑ “Quand tu trouves cet élément XML → applique cette règle”

Structure de base d'un fichier XSLT

</> XML

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

  <xsl:template match="/">
    <!-- point de départ -->
  </xsl:template>

</xsl:stylesheet>
```

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

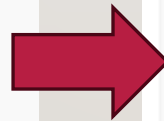
- Exemple complet



XML :

</> XML

```
<students>
  <student>
    <name>Ali</name>
    <age>22</age>
  </student>
</students>
```



XSLT :

</> XML

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>

      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>

    </body>
  </html>
</xsl:template>
```

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

- **Explication du template XSLT:**

- ❑ template = une règle de transformation
- ❑ match="/" = **point de départ du document XML**
- ❑ “Commence le traitement depuis la racine du XML”

XSLT :

</> XML

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>

      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>

    </body>
  </html>
</xsl:template>
```

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

- Explication du template **XSLT**:

- ❑ On crée directement du **HTML**
- ❑ Le résultat final sera une page web
- ❑ Donc XSLT est en train de transformer XML → HTML

XSLT :

</> XML

```
<xsl:template match="/">
```

```
  <html>
```

```
    <body>
```

```
      <h2>Students List</h2>
```

```
      <xsl:for-each select="students/student">
```

```
        <p>
```

```
          <xsl:value-of select="name"/> -
```

```
          <xsl:value-of select="age"/>
```

```
        </p>
```

```
      </xsl:for-each>
```

```
    </body>
```

```
  </html>
```

```
</xsl:template>
```


I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

- Explication du template **XSLT**:

- ☐ `xsl:for-each` = boucle

- ☐ `select="students/student"` = XPath

Cela signifie :

- ☐ “Pour chaque élément `<student>` dans `<students>`”

XSLT :

```
</> XML
```

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>
      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>
    </body>
  </html>
</xsl:template>
```

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

Exemple XML correspondant :

</> XML

```
<students>
  <student>...</student>
  <student>...</student>
</students>
```

</> XML

```
<p>
```

Pour chaque étudiant, on crée un paragraphe HTML

XSLT :

</> XML

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>
      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>
    </body>
  </html>
</xsl:template>
```

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

- **Explication du template XSLT:**

- value-of = affiche une valeur
- select="name" = prend la balise
<name>

Résultat :

- affiche le nom de l'étudiant

XSLT :

```
</> XML

<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>

      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>

    </body>
  </html>
</xsl:template>
```

I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

Explication du template XSLT:

- Juste un séparateur texte (affiché tel quel)
- Affiche l'âge de l'étudiant
- Fin du paragraphe pour un étudiant
- Fin de la boucle
On répète pour chaque étudiant
- Fin de la structure HTML
- Fin de la règle XSLT

XSLT :

`</> XML`

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>

      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>

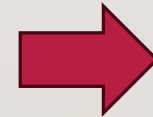
    </body>
  </html>
</xsl:template>
```


I. XSLT (XSL TRANSFORMATIONS) — LE CŒUR

Si ton XML est :

</> XML

```
<students>
  <student>
    <name>Ali</name>
    <age>22</age>
  </student>
  <student>
    <name>Sara</name>
    <age>21</age>
  </student>
</students>
```



Le HTML généré sera :

</> HTML

```
<h2>Students List</h2>

<p>Ali - 22</p>
<p>Sara - 21</p>
```

INSTRUCTIONS IMPORTANTES EN XSLT

Instruction	Rôle
<code><xsl:value-of></code>	afficher une valeur
<code><xsl:for-each></code>	parcourir des éléments
<code><xsl:if></code>	condition
<code><xsl:choose></code>	alternative (if/else)
<code><xsl:apply-templates></code>	appliquer des templates

II. XPATH — LE MOTEUR DE SÉLECTION

- XSLT utilise **XPath** pour cibler les éléments XML.

XPath = langage de navigation dans XML

Exemples :

Expression	Signification
<code>/students/student</code>	tous les étudiants
<code>//name</code>	tous les noms
<code>student[age > 20]</code>	étudiants > 20 ans

II. XPATH — LE MOTEUR DE SÉLECTION

- XPath permet de dire **quoi transformer**

XSLT :

</> XML

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Students List</h2>
      <xsl:for-each select="students/student">
        <p>
          <xsl:value-of select="name"/> -
          <xsl:value-of select="age"/>
        </p>
      </xsl:for-each>
    </body>
  </html>
</xsl:template>
```


III. XSL-FO (FORMATTING OBJECTS)

- Utilisé pour :

- ☐ générer des **PDF**

- ☐ contrôler la **mise en page avancée**

Exemple :

`</>` XML

Important :

- ❖ Moins utilisé aujourd'hui que XSLT
- ❖ Souvent remplacé par des outils modernes
- ❖ Mais encore utilisé dans les systèmes d'impression

```
<fo:block font-size="12pt">
```

Bonjour le monde

```
</fo:block>
```

PROCESSUS COMPLET DE TRANSFORMATION

- Étapes :

1. XML (données)
2. XSLT (règles)
3. Processeur XSLT
4. Résultat (HTML / PDF / XML)

- Schéma simple :

XML + XSLT → Processeur XSLT → Résultat (HTML)

RÉSUMÉ

XSL = système complet :

Partie	Rôle
XSLT	Transformer
XPath	Sélectionner
XSL-FO	Formatter

CHAPITRE 04: GALAXIE XML

4.3.Applications XML : RDF, SVG, ...

4.3.APPLICATIONS XML : RDF, SVG, ...

- XML n'est pas juste un format de données, c'est une **base pour plusieurs technologies** utilisées dans différents domaines.
 1. RDF (Resource Description Framework)
 2. SVG (Scalable Vector Graphics)

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

➤ Définition

RDF est un modèle basé sur XML qui sert à **décrire des informations et leurs relations**.

- ✓ Pas juste stocker des données...
- ✓ mais dire **qui est lié à quoi**

➤ Très utilisé dans :

- le **web sémantique**
- les moteurs de recherche
- les bases de connaissances

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- RDF fonctionne toujours avec :

Sujet — Prédicat — Objet

C'est ça RDF

- **Exemple 1:**

Phrase : “Ali est étudiant”

- On la découpe :

Élément	Valeur
Sujet	Ali
Prédicat	est
Objet	étudiant

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Exemple 2:**

“Sara habite à Alger”

Sujet	Prédicat	Objet
Sara	habite	Alger

RDF transforme les phrases en **relations structurées**

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Pourquoi on fait ça ?**

Pour que les machines comprennent :

- ☐ qui est qui
- ☐ qui fait quoi
- ☐ les relations entre les données

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Visualisation**
- RDF = **graphe**
- Données = réseau (graph)

Explication

- [Ali] = un nœud (une entité)
- [étudiant] = un autre nœud
- — est —> = une relation (flèche)

Ali — est —> étudiant
Sara — habite —> Alger

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Pourquoi on appelle ça “graphe” ?**

Parce que tu peux ajouter plein de relations :

```
[Ali] — est —▶ [étudiant]  
[Ali] — habite —▶ [Alger]  
[Sara] — aime —▶ [math]
```

Ça devient un **réseau connecté**

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Donc “Données = réseau” ça veut dire :**
 - Au lieu de stocker :
 - ❑ juste des lignes (comme Excel)
 - ❑ ou des tables (comme base de données)
 - RDF stocke :
 - ❑ **des relations connectées entre données**

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- Comparaison simple:

A. Base de données classique :

Nom	Ville
Ali	Alger

B. RDF (graphe) :

Ali — habite → Alger

Plus flexible et plus “intelligent”

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Pourquoi c'est puissant ?**

- Parce que tu peux :

- ☐ relier les données facilement

- ☐ ajouter des relations sans casser le système

- ☐ naviguer comme un réseau (comme Google)

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Exemple réel**

- Si tu cherches :
- “Étudiants qui habitent à Alger”

- RDF peut suivre les flèches :

Ali — habite → Alger
Ali — est → étudiant

- ✓ Donc Ali correspond

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- Résumé:

- ❑ RDF = des **points (objets)**

- ❑ reliés par des **flèches (relations)**

- ❑ ça forme un **réseau (graphe)**

RDF, c'est comme un réseau social :

chaque personne est connectée à d'autres par des relations.

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- Et XML dans tout ça ?
- RDF peut être écrit en XML (RDF/XML)
- Mais:
 - ✓ XML = format
 - ✓ RDF = logique (relations)

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- Exemple simple en RDF/XML :

```
</> XML
```

```
<rdf:Description rdf:about="Ali">  
  <type>Student</type>  
</rdf:Description>
```

- Ça veut dire :

Ali → type → Student

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Résumé :**

- **RDF =**

- ✓ des phrases
- ✓ transformées en relations
- ✓ sous forme de graphe

RDF, c'est comme transformer des phrases normales en relations que la machine peut comprendre.

4.3.1. RDF (RESOURCE DESCRIPTION FRAMEWORK):

- **Résumé :**

- A. XML :**

- C'est un **format**
- Sert à **structurer des données**

- B. RDF :**

- C'est un **modèle de données**
- Sert à **exprimer des relations (graphe)**

Exemple :

Ali — est — étudiant

```
</> XML
```

```
<student>
```

```
    <name>Ali</name>
```

```
</student>
```


2. SVG (SCALABLE VECTOR GRAPHICS)

➤ Définition:

SVG est un langage basé sur XML qui sert à créer des images **vectorielles**.

4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ **C'est quoi "vectoriel" ?** Il y a 2 types d'images :

1. Image classique (JPG, PNG)

- ❑ faite de **pixels**
- ❑ si tu zoomes → ça devient flou

2. **SVG (vectoriel)**

- ❑ faite de **formes mathématiques** (cercles, lignes...)
- ❑ si tu zoomes → reste parfait

4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

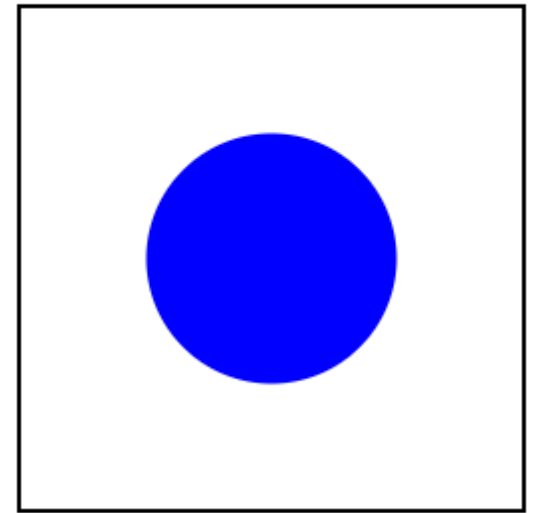
➤ Pourquoi **SVG** utilise **XML** ?

Parce que chaque élément graphique est écrit comme une balise XML

➤ Exemple:

```
</> HTML  
  
<html>  
<body>  
  
  <svg width="200" height="200">  
    <circle cx="100" cy="100" r="50" fill="blue"/>  
  </svg>  
  
</body>  
</html>
```

Cercle SVG



4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ Explication

❑ `<svg>` → conteneur principal

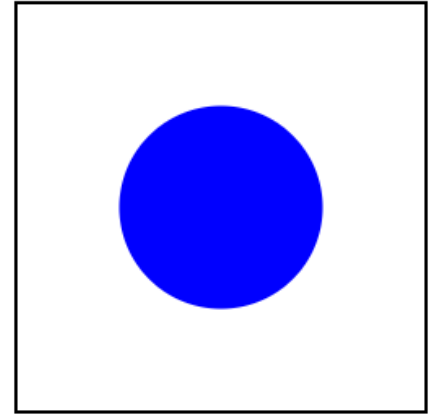
❑ `<circle>` → dessine un cercle

❑ `cx, cy` → position

❑ `r` → rayon

❑ `fill` → couleur

Cercle SVG



```
</> HTML

<html>
<body>

<svg width="200" height="200">
  <circle cx="100" cy="100" r="50" fill="blue"/>
</svg>

</body>
</html>
```


4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ Les formes principales en SVG:

Balise	Rôle
<code><circle></code>	cercle
<code><rect></code>	rectangle
<code><line></code>	ligne
<code><ellipse></code>	ellipse
<code><polygon></code>	forme complexe

4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ Pourquoi **SVG** est puissant ?

- ☐ Modifiable avec du code
- ☐ Léger (petite taille)
- ☐ Compatible web
- ☐ Animable (avec CSS/JS)

4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ Utilisations réelles :

- ❑ SVG est utilisé pour
- ❑ graphiques (charts)
- ❑ cartes
- ❑ logos
- ❑ interfaces web

4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ **Lien avec XML**

□ SVG est :

- ✓ écrit en XML
- ✓ structuré comme XML
- ✓ lisible comme XML

4.3.2. SVG (SCALABLE VECTOR GRAPHICS)

➤ **Résumé final**

SVG = XML (structure)

+

instructions graphiques (dessin)

CHAPITRE 04: GALAXIE XML

4.4. Traitement: DOM et SAX / Processing XML: DOM and SAX

On parle de **comment lire et traiter un fichier XML** dans un programme.

This part explains **how a program reads and processes an XML file**

4.4.TRAITEMENT: DOM ET SAX

➤ Le problème:

Tu as un fichier XML :

```
<? XML  
  
<students>  
  <student>  
    <name>Ali</name>  
    <age>22</age>  
  </student>  
</students>
```

❑ Question :

Comment un programme lit ce fichier ?

4.4.TRAITEMENT: DOM ET SAX

- Les deux principales interfaces de programmation XML :
 1. DOM (Document Object Model), basé sur une représentation hiérarchique
 2. SaX (Simple API for XML), basé sur des déclencheurs (événements/action)

4.4.TRAITEMENT: DOM ET SAX

➤ I. DOM (Document Object Model)

- Principe:

Le programme charge **tout le XML** en mémoire et le transforme en **arbre**

Visualisation

```
students
├── student
│   ├── name → Ali
│   └── age → 22
```

4.4.1. DOM (DOCUMENT OBJECT MODEL)

➤ **Avantages:**

- ❖ Facile à comprendre
- ❖ Navigation simple
(comme un arbre)
- ❖ On peut modifier les données

➤ **Advantages:**

- ❖ Easy to understand
- ❖ Easy to navigate
- ❖ You can modify the data

4.4.1. DOM (DOCUMENT OBJECT MODEL)

➤ Inconvénients

❖ Plus difficile à programmer

❖ Pas de modification facile

➤ Disadvantages

❖ Uses a lot of memory

❖ Slow for large files

4.4.1. DOM (DOCUMENT OBJECT MODEL)

➤ **Idée simple**

DOM = “je charge tout et je travaille dessus”

➤ **Simple idea**

DOM = “load everything, then work on it”

4.4.2. SAX (SIMPLE API FOR XML)

➤ Principe:

- ❑ Le fichier est lu **ligne par ligne**
- ❑ comme un flux (stream)

➤ Idea

- ❑ The program reads the XML **line by line**
- ❑ like a stream

4.4.2. SAX (SIMPLE API FOR XML)

- **Comment ça marche ?**

➤ Le programme réagit à des **événements** :

- ✓ début d'une balise
- ✓ fin d'une balise
- ✓ contenu

- **How it works**

- SAX uses **events**

When it reads:

```
<name>Ali</name>
```

- it generates events:
 - ✓ start element → name
 - ✓ text → Ali
 - ✓ end element → name

4.4.2. SAX (SIMPLE API FOR XML)

➤ **Avantages**

- ❖ Très rapide
- ❖ Très léger (peu de mémoire)
- ❖ idéal pour gros fichiers

➤ **Advantages**

- ❖ Very fast
- ❖ Uses little memory
- ❖ Good for large files

4.4.2. SAX (SIMPLE API FOR XML)

➤ Inconvénients

- ❖ Plus difficile à programmer
- ❖ Pas de modification facile

➤ Disadvantages

- ❖ Harder to program
- ❖ Cannot easily modify data

4.4.2. SAX (SIMPLE API FOR XML)

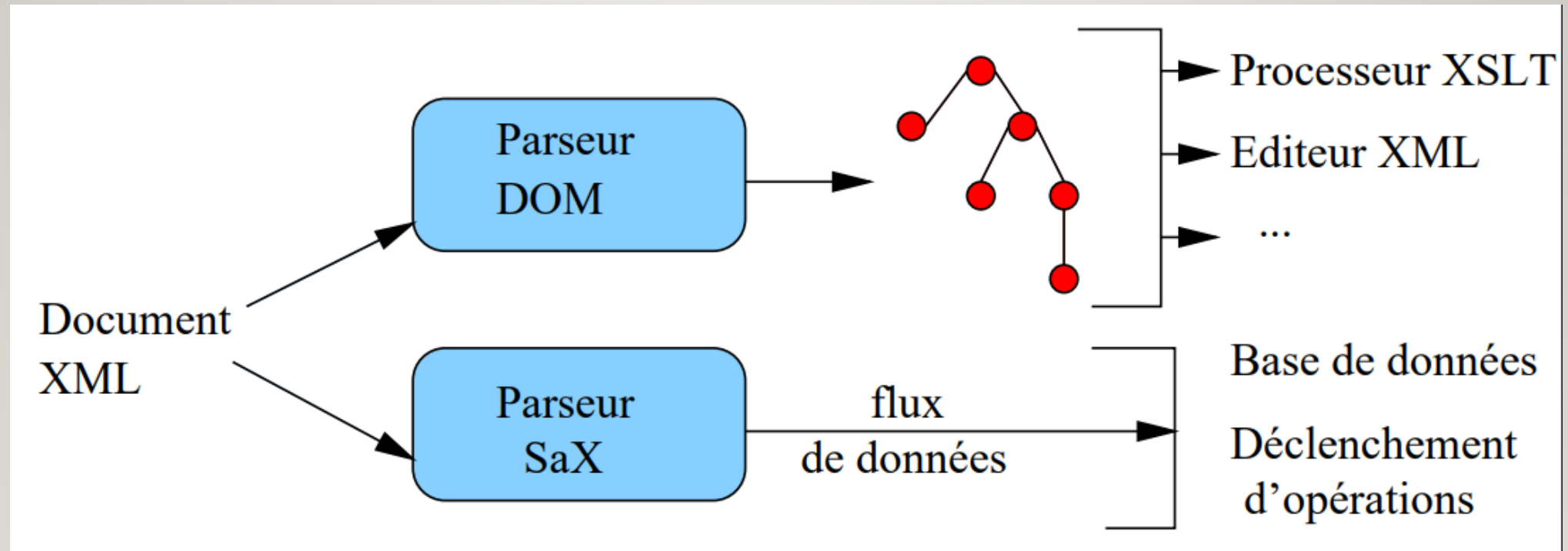
➤ **Idée simple**

SAX = “je lis petit à petit et je réagis”

➤ **Simple idea**

SAX = “read step by step and react”

4.4.TRAITEMENT: DOM ET SAX



4.4. COMPARAISON SIMPLE / SIMPLE COMPARISON:

Critère	DOM	SAX
Lecture	tout d'un coup	ligne par ligne
Mémoire	élevée	faible
Facilité	facile	difficile
Modification	oui	non

Feature	DOM	SAX
Reading	all at once	step by step
Memory	high	low
Ease	easy	harder
Modify data	yes	no

4.4. RÉSUMÉ FINAL / FINAL SUMMARY:

❑ DOM = comme ouvrir tout un livre

❑ SAX = comme lire ligne par ligne

❖ DOM = arbre en mémoire

❖ SAX = lecture en flux

❑ DOM = reading a whole book

❑ SAX = reading line by line

❖ DOM = tree in memory

❖ SAX = stream processing

4.4.TRAITEMENT: DOM ET SAX

➤ Simple Python Example:
XML file (students.xml)

```
<students>
  <student>
    <name>Ali</name>
    <age>22</age>
  </student>
  <student>
    <name>Sara</name>
    <age>21</age>
  </student>
</students>
```

4.4.TRAITEMENT: DOM ET SAX

A. DOM Example (easy)

Using xml.dom.minidom

Output:

Ali 22
Sara 21

`</> Python`

```
from xml.dom import minidom
```

```
# Load XML
```

```
doc = minidom.parse("students.xml")
```

```
# get all students
```

```
students = doc.getElementsByTagName("student")
```

```
for s in students:
```

```
    name = s.getElementsByTagName("name")[0].firstChild.data
```

```
    age = s.getElementsByTagName("age")[0].firstChild.data
```

```
    print(name, age)
```

❑ The whole XML is loaded into memory

❑ You navigate like a tree

4.4.TRAITEMENT

B. SAX Example (event-based)

Using xml.sax

- ❑ Reads XML step by step
- ❑ Reacts to events (start, text, end)

```
</> Python

import xml.sax

class StudentHandler(xml.sax.ContentHandler):

    def __init__(self):
        self.current = ""
        self.name = ""
        self.age = ""

    def startElement(self, tag, attrs):
        self.current = tag

    def characters(self, content):
        if self.current == "name":
            self.name = content
        elif self.current == "age":
            self.age = content

    def endElement(self, tag):
        if tag == "student":
            print(self.name, self.age)

# run parser
parser = xml.sax.make_parser()
parser.setContentHandler(StudentHandler())
parser.parse("students.xml")
```

4.4.TRAITEMENT: DOM ET SAX

➤ Key Difference

❑ DOM → easier, but heavy

❑ SAX → faster, but more complex

➤ Extracting XML with Python:

Python has **many ways** to do that:

❖ DOM

❖ SAX

❖ ElementTree (very common)

4. GALAXIE XML

4.5. LES POINTEURS: XPOINTER POINTERS: XPOINTER

4.5. LES POINTEURS: XPOINTER

➤ I. Définition / Definition :

XPointer (XML Pointer Language) permet de :

- pointer vers une partie précise d'un document **XML**

- is used to:

point to a specific part of an XML document

4.5. LES POINTEURS: XPOINTER

- **Idée simple**

- ❑ XML = un grand document

- ❑ XPointer = un moyen de dire :

- “Je veux aller exactement ici ”

- **Simple idea :**

- ❑ XML = a big document

- ❑ XPointer = a way to say:

- “I want to go exactly here”

4.5. LES POINTEURS: XPOINTER

- 2. Exemple simple:

`</> XML`

```
<students>
  <student id="s1">
    <name>Ali</name>
  </student>
  <student id="s2">
    <name>Sara</name>
  </student>
</students>
```

Objectif

Accéder

seulement à :

`</> XML`

```
<student id="s2">
  <name>Sara</name>
</student>
```

XPointer

```
#xpointer(//student[@id='s2'])
```


4.5. LES POINTEURS: XPOINTER

➤ 3. Relation with XPath:

- Very important: XPointer uses **XPath**

• 4. Why use XPointer?

- ☐ Access a specific part of XML
- ☐ Create links to parts of a document
- ☐ Often used with XLink

XPath:

```
//student[@id='s2']
```

XPointer:

```
#xpointer(//student[@id='s2'])
```

4.5. LES POINTEURS: XPOINTER

- 5. Example : Like a link

```
file.xml#xpointer(//student[1])
```

- This means:

Open the file and go directly to the first student

4.5. LES POINTEURS: XPOINTER

➤ 6. Simple comparison

Like in HTML:

HTML → #section I

- XML → #xpointer(...)

4.5. LES POINTEURS: XPOINTER

➤ Final summary:

XPointer = a way to point to a specific part inside an XML document

➤ Tip

- **XPath** = used to find elements
- XPointer** = used to point to an element (like a link)

4.5. LES POINTEURS: XPOINTER

➤ Key Difference:

- ❑ XPath = search (query)
- ❑ XPointer = point (link to a location)

4.5. LES POINTEURS: XPOINTER

I. XPath → to find elements

XPath is used inside a program (Python, XSLT...)

- **Example:**

```
//student[@id='s2']
```

Important:

- XPath **returns a result**
- It is used to **process data**

- This means: “Find the student with id = s2”

4.5. LES POINTEURS: XPOINTER

- **2. XPointer** → to create a link

Important:

XPointer is used in a **URL or link**

- XPointer is used for **navigation**
- Like an HTML link

- **Example:**

```
file.xml#xpointer(//student[@id='s2'])
```

- This means: “Open the file AND go directly to this location”

4.5. LES POINTEURS: XPOINTER

Simple Comparison:

➤ XPath

- Like asking a question:

“Give me this element”

➤ XPointer

- Like clicking a link:

“Take me directly here”

4. GALAXIE XML

4.6. LES LIENS: XLINK LINKS: XLINK

4.6. LINKS: XLINK

➤ I. Definition:

- **XLink (XML Linking Language)** is used to:
- create links between **XML** documents or parts of documents

➤ Simple idea

- Like in HTML:

```
</> HTML
```

```
<a href="page.html">Link</a>
```

- ## ➤ In XML, we use **XLink** to create similar links

4.6. LINKS: XLINK

➤ 2. Why do we need XLink?

- Because XML does not have links by default
- XLink adds the ability to:
 - ❑ connect documents
 - ❑ navigate between data
 - ❑ build relationships

4.6. LINKS: XLINK

➤ 3. Simple Example:

```
</> XML
```

```
<book xmlns:xlink="http://www.w3.org/1999/xlink"  
      xlink:href="book2.xml"  
      xlink:type="simple">
```

```
  Go to another book
```

```
</book>
```

- **Meaning:**

This creates a link to **book2.xml**

4.6. LINKS: XLINK

➤ 4.Types of Xlink :

A. Simple Link (most important)

Like HTML <a>

```
</> XML
```

```
xlink:type="simple"
```

```
xlink:href="file.xml"
```


4.6. LINKS: XLINK

➤ 4.Types of Xlink :

B. Extended Link (advanced)

Can link **multiple resources together**

```
</> XML
```

```
<links xmlns:xlink="http://www.w3.org/1999/xlink">
```

```
  <!-- Define resources -->
```

```
  <book1 xlink:type="resource" xlink:label="b1">
```

```
    Book 1
```

```
  </book1>
```

```
  <book2 xlink:type="resource" xlink:label="b2">
```

```
    Book 2
```

```
  </book2>
```

```
  <!-- Define a link between them -->
```

```
  <connection xlink:type="arc"
```

```
    xlink:from="b1"
```

```
    xlink:to="b2"/>
```

```
</links>
```

4.6. LINKS: XLINK

➤ 5. Relation with XPointer :

- **XLink** → creates the link
- **XPointer** → defines where the link goes

Example:

```
</> XML
```

```
xlink:href="file.xml#xpointer(//student[1])"
```

Go to file.xml and directly to the first student

4.6. LINKS: XLINK

➤ 6. Simple Comparison:

Technology	Role
XPath	find elements
XPointer	point to location
XLink	create links

4.6. LINKS: XLINK

➤ Final Summary:

- ❑ XLink is like HTML links, but for XML documents.
- ❑ a way to create links between XML resources
- ❑ XLink connects XML documents, just like hyperlinks connect web pages.